

ВВЕДЕНИЕ

Цель – исследовать тему применения гео-структур в PostgreSQL, расширив функционал системы бронирования билетов.

Воркфлоу системы до расширения: Пользователь выбирает событие → видит схему зала с доступными местами → выбирает места → система временно резервирует их (5 минут) → пользователь подтверждает → система переводит бронь в «оплачен» и генерирует билет.

Основные задачи:

- Определить какие бывают гео-структуры, где они используются, как реализуются, их особенности
- Выбрать конкретный тип гео-структур и исследовать его применение в PostgreSQL
- Реализовать расширение системы, используя выбранный тип гео-структур
- Проанализировать результаты, сделать выводы

1. Проблема

Ранее был спроектирован и реализован сервис бронирования билетов в театры. Также была разработана OLTP-схема базы данных в PostgreSQL, сгенерированы тестовые данные и проанализированы планы выполнения запросов. Однако существующая реализация не поддерживает пространственные запросы – например, поиск ближайших театров по геолокации пользователя.

Предложение:

При заходе в приложение пользователь может найти театры, расположенные в радиусе 10 км от его текущего местоположения или посмотреть какие находятся ближе всех.

Сценарий использования:

Сценарий включает два основных варианта использования:

1. «Показать все театры в радиусе R км» – пользователь указывает радиус и хочет увидеть полный список подходящих театров.
2. «Показать N ближайших театров» – пользователь хочет быстро увидеть самые близкие театры, не указывая радиус (N задается программно).

Детали сценария:

- Пользователь открывает карту или список театров.
- Приложение определяет геолокацию пользователя (или пользователь вводит адрес).
- В зависимости от выбранного режима приложение отправляет запрос к одному из API
- API возвращает список театров, отсортированных по расстоянию.

Какой обработчик API, сценарий или таблицы затрагиваются

Старые обработчики API никак не затрагиваются, реализуются два новых эндпоинта:

GET /venues/nearby – поиск в радиусе,

GET /venues/closest – поиск KNN (ближайшие).

Для новых эндпоинтов создается таблица venue, в которой указываются id, название, город, адрес, местоположение театров.

Сколько данных или запросов ожидается

Параметр	Значение
Количество театров	1 000 записей
Расширение количества театров	до 100 000 записей
Объём записей	15 Мб + 6 Мб (индексы) = 21 Мб
Частота запросов к API	до 100 запросов в секунду (в часы пик) или до 360 000 в час пик
Доля запросов KNN	~60%
Доля запросов в радиусе	~40%

Целевая задержка

Запрос геопоиска выполняется при загрузке главного экрана приложения – это первый сценарий взаимодействия пользователя с сервисом. Любая задержка на этом этапе напрямую влияет на восприятие работы всего приложения: пользователь ожидает увидеть список ближайших театров практически мгновенно. Поэтому время выполнения запроса должно укладываться в единицы миллисекунд (менее 10 мс), а допустимый максимум не должен превышать 50 мс.

Текущее ограничение

Без специализированного пространственного индекса фильтрация по координатам потребовала бы вычисления расстояния для каждой строки таблицы (что также бы занимало время), что при росте таблицы до десятков/сотен тысяч строк привело бы к увеличению времени отклика до сотен миллисекунд и выше.

2. Гео-структуры и их особенности

Прежде чем рассматривать конкретные структуры, важно понимать общую проблему: классический индекс B-tree строит полный порядок по одному значению (число, строка, дата) – для любых двух значений можно сказать, какое больше. Для точек на плоскости (x, y) такого порядка не существует: точка $(1, 100)$ не «больше» и не «меньше» точки $(50, 2)$ в каком-то однозначном смысле, который сохранял бы пространственную близость при упорядочивании.

Поэтому все пространственные структуры данных решают одну и ту же задачу – как сгруппировать близкие в пространстве объекты так, чтобы при поиске можно было быстро отбросить неподходящие группы, не проверяя их по отдельности.

Различия между структурами – это различия в способе разбиения пространства на группы и в том, пересекаются ли эти группы.

Ниже рассмотрены основные виды таких структур.

2.1. R-Tree (дерево прямоугольников)

R-Tree организует объекты в иерархию минимальных ограничивающих прямоугольников (MBR). Каждый лист дерева хранит реальные объекты (точки, линии, полигоны) вместе с прямоугольником, который их полностью охватывает. Каждый внутренний узел хранит прямоугольник, охватывающий MBR всех своих дочерних узлов, и так до корня – корень охватывает вообще все объекты в дереве.

Ключевая особенность: MBR на одном уровне дерева могут перекрываться. Это происходит потому, что при вставке нового объекта алгоритм выбирает поддереву, чей MBR требует наименьшего расширения – но если объект физически находится на стыке двух областей, расширения MBR разных поддеревьев могут начать перекрываться.

Алгоритм поиска

При поиске объектов, пересекающихся с заданной областью запроса (например, прямоугольником или окружностью), алгоритм спускается от корня:

для каждого узла проверяется, пересекается ли его MBR с областью запроса. Если нет – всё поддерево отбрасывается целиком, без захода внутрь. Если да – рекурсивно проверяются дети. На листовом уровне выполняется точная проверка геометрии (не просто MBR, а реальная форма объекта).

Из-за возможных перекрытий MBR один и тот же объект запроса может потребовать спуска в несколько поддеревьев одновременно.

Поддержка KNN

R-Tree поддерживает поиск ближайших соседей (K Nearest Neighbors): алгоритм поддерживает приоритетную очередь узлов, упорядоченную по минимально возможному расстоянию от точки запроса до MBR узла, и обходит дерево, на каждом шаге раскрывая узел с наименьшей нижней границей расстояния. Это позволяет находить N ближайших объектов без необходимости заранее задавать радиус поиска и без сканирования всех данных.

Реализация в PostgreSQL

R-Tree – это основная структура, реализованная в PostgreSQL для геоданных, доступная через индекс GiST, предоставляемым PostGIS.

GiST (Generalized Search Tree) – это не сама структура R-Tree, а обобщённый framework для построения сбалансированных деревьев поиска с произвольными определяемыми пользователем функциями сравнения, объединения и проверки пересечения областей.

GiST/R-Tree в PostgreSQL поддерживает два типа данных PostGIS – geometry и geography.

Основные операторы: && (пересечение MBR) – главный оператор, который использует индекс; <-> (расстояние) – для KNN-поиска ближайших соседей без задания радиуса; ST_DWithin() – главная функция для поиска в радиусе, внутри применяет &&.

2.2. Quad-tree (квадродерево)

Квадродерево рекурсивно делит двумерное пространство на 4 равные непересекающиеся квадранта. Если в каком-то квадранте оказывается больше объектов, чем заданный порог, этот квадрант снова делится на 4 части, и так

далее, пока в каждом листовом квадранте не останется приемлемое количество объектов. Получается дерево, где каждый узел соответствует определённой прямоугольной области пространства, и любая точка однозначно попадает в одну цепочку узлов от корня к листу.

Особенности

В отличие от R-Tree, разбиения не перекрываются – это упрощает определение, к какому узлу относится точка (нужно просто сравнить координаты с границами квадрантов). Однако у этого есть обратная сторона: если данные сильно неравномерны (например, плотное скопление точек в одном углу пространства), дерево в этой области становится очень глубоким, тогда как в пустых областях – почти пустым. Следовательно, балансировка такого дерева сложнее, чем у R-Tree.

Реализация в PostgreSQL

Реализуется через SP-GiST – framework, аналогичный GiST, но предназначенный для структур с непересекающимися разбиениями

Поддерживаемые операторы: проверка пересечения и вложенности – `&&`, `@>`, `<@`, `~` и аналогичные; `ST_DWithin`, `ST_Intersects` и другие.

Не поддерживается оператор `<->` и, соответственно, KNN-поиск (`ORDER BY geom <-> point LIMIT N`).

2.3 Kd-tree (k-мерное дерево)

Kd-tree – бинарное дерево, в котором на каждом уровне пространство делится по одной координатной оси, попеременно (на первом уровне по X, на втором по Y, на третьем снова по X, и так далее). Каждый узел хранит одну точку (медианную), разделяющую гиперплоскость, а также ссылки на правое и левое поддеревья; левое поддерево содержит точки с одной стороны гиперплоскости, правое – с другой.

Особенности

Kd-tree очень эффективен для статических наборов данных – построение оптимального сбалансированного дерева требует сортировки данных, и при добавлении/удалении точек дерево может разбалансироваться, требуя

периодической перестройки. Это делает структуру менее удобной для OLTP-нагрузок с частыми вставками/обновлениями, характерными для реляционных СУБД.

Реализация в PostgreSQL

Kd-tree как самостоятельная индексная структура для геометрических типов в PostgreSQL/PostGIS не реализована.

2.4. Grid index (сеточный индекс)

Простейший подход: пространство разбивается на ячейки фиксированного размера (сетку), каждому объекту присваивается идентификатор ячейки (или нескольких ячеек, если объект пересекает границы). Частный случай – геохеш: координаты точки кодируются в строку фиксированной длины путём чередующегося деления диапазонов широты и долготы пополам; точки, находящиеся близко друг к другу, как правило получают строки с общим префиксом.

Особенности

Главное достоинство – простота: идентификатор ячейки/геохеш можно индексировать стандартным B-tree индексом, без специализированных пространственных индексов.

Главный недостаток: две точки, находящиеся физически очень близко друг к другу, но по разные стороны границы ячейки/префикса, получают совершенно разные идентификаторы.

Реализация в PostgreSQL

Grid-индекс не является встроенным типом индекса PostgreSQL и не предоставляется PostGIS как отдельная индексная структура.

Итоговая сводная таблица:

Структура	Принцип разбиения	Перекрытия	KNN	Реализация в PostgreSQL
R-Tree	Иерархия MBR	Да, допускаются	Да	GiST (полная реализация для geometry/geography)
Quad-tree	Рекурсивное деление на 4 квадранта	Нет	Нет	SP-GiST (частичная – без KNN)
Kd-tree	Деление по одной оси попеременно	Нет	Нет	Не реализована
Grid / geohash	Фиксированная сетка / геохеш	Нет (границы)	Нет	Не реализована

Вывод по выбору структуры

Для задачи геопоиска площадок в сервисе бронирования билетов требуются оба сценария – «объекты в радиусе» и «N ближайших объектов» (KNN). Из всех рассмотренных структур только R-Tree (через GiST) поддерживает оба сценария одновременно для типов geometry и geography. Квадродерево (SP-GiST) покрывает только первый сценарий, kd-tree и grid/geohash не имеют полноценной встроенной реализации в PostgreSQL/PostGIS для основных геометрических типов.

3. GiST и R-tree

Основные типы данных PostGIS

Для хранения координат PostGIS предоставляет два типа данных:

`geometry` – представляет данные в декартовой системе координат. Вычисления расстояний и площадей выполняются в единицах исходной системы координат – обычно это градусы (SRID 4326). Вычисления быстрее, так как не требуют учёта формы Земли.

`geography` – представляет данные на сферической модели Земли. Все вычисления расстояний и площадей автоматически учитывают кривизну планеты и возвращаются в метрах. Вычисления более ресурсоёмкие, но единицы измерения сразу удобны для практического использования (вывод пользователю, задание радиуса в метрах).

В обоих случаях точка (координаты местоположения) задаётся как `Point` с указанием системы координат через SRID:

```
GEOMETRY(Point, 4326)  
GEOGRAPHY(Point, 4326)
```

SRID

SRID – числовой идентификатор системы координат, используемый для интерпретации числовых значений координат, хранящихся в геометрии. Один и тот же набор чисел (например, (37.6, 55.7)) может означать разные точки на Земле в зависимости от того, какая система координат применяется для их интерпретации.

Наиболее распространённый SRID – 4326, соответствующий системе координат, используемой GPS-устройствами. В этой системе координаты задаются как пары (долгота, широта) в градусах.

Для типа `geometry` SRID необходимо указывать явно при создании точки:

```
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)
```

Для типа `geography` SRID 4326 принимается по умолчанию, поэтому достаточно:

```
ST_MakePoint(37.6173, 55.7558)::geography
```

Важно заметить, что операции между геометриями с разными SRID без явного приведения вызывают ошибку (Operation on mixed SRID geometries).

Вычисление расстояний

Если используется geometry с SRID 4326, то координаты точек – это пары (долгота, широта) в градусах, и PostgreSQL интерпретирует их как точки на декартовой плоскости, без какого-либо учёта того, что это координаты на сфере.

Функция ST_Distance для geometry вычисляет обычное евклидово расстояние между двумя точками. Результат – число в градусах.

У этого есть две проблемы, которые дают высокую погрешность:

- Разбиение по долготе и широте неравномерное (чем ближе к экватору, тем больше «сетка»)

- Результат ST_Distance для geometry нельзя напрямую интерпретировать как физическое расстояние – это число, отражающее «удалённость» от точки в декартовой системе координат.

Именно поэтому для расчётов реальных расстояний в метрах используется тип geography: для него ST_Distance вычисляет расстояние, учитывая округлость Земли, что даёт корректный результат в метрах, но требует больше вычислительных ресурсов.

Создание индекса

```
CREATE INDEX idx_venues_location ON venues USING GIST (location);
```

Эта команда создаёт R-Tree-индекс: PostGIS реализует внутри GiST-фреймворка логику построения иерархии MBR для значений столбца location.

Стоит отметить, что индекс GiST строится под конкретный тип столбца. Если тип столбца geography, то и индекс должен быть сделан на тип geography. Иначе индекс не будет использован, а PostgreSQL выполнит полный перебор таблицы (Seq Scan).

Используемые функции и операторы

В рамках исследования применялись следующие основные функции и операторы PostGIS:

`ST_MakePoint(lon, lat)` – создаёт точку из координат долготы и широты.

`ST_SetSRID(geom, srid)` – задаёт систему координат для геометрии.

`ST_Distance(a, b)` – вычисляет расстояние между двумя объектами. Не использует индекс GiST, поэтому применяется не для фильтрации, а для вывода точных значений.

`ST_DWithin(a, b, distance)` – проверяет, находится ли объект `a` в пределах заданного расстояния `distance` от объекта `b`. Это основная функция для сценария «объекты в радиусе». Она поддерживается индексом GiST. Внутри использует оператор `&&`.

`&&` – оператор пересечения bounding box'ов (MBR). Базовый оператор, который GiST/R-Tree использует для отсекающих неподходящих поддеревьев при выполнении большинства пространственных запросов.

`<->` – оператор расстояния, обеспечивающий поддержку KNN. Вычисляет значение приближенно (от MBR к MBR), поэтому не точнее `ST_Distance`. Используется в конструкции:

```
ORDER BY location <-> point LIMIT N
```

GiST обходит дерево в порядке возрастания расстояния от точки запроса и останавливается после получения LIMIT строк, не сканируя всю таблицу.

4. Реализация расширения системы с применением GiST

Для контейнеризации приложения будет использован docker desktop.

Для работы с индексом GiST необходимо установить расширение PostGIS.

Его можно установить через docker-compose.

docker-compose выглядит так:

```
version: '3.8'

services:
  postgres:
    image: postgis/postgis:16-3.4
    container_name: postgres
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - ./pgdata:/var/lib/postgresql/data
```

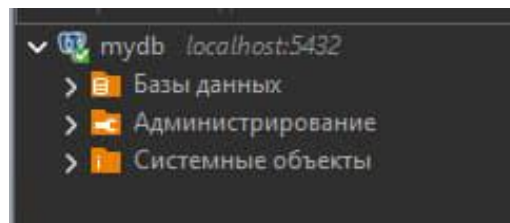
Запуск докера:

```
PS C:\Users\Andrey\Desktop\System_booking\docker> docker-compose up -d
time="2026-06-11T22:16:05+03:00" level=warning msg="C:\\Users\\Andrey\\Desktop\\System_booking\\docker\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 21/21
✓Image postgis/postgis:16-3.4 Pulled 11.3s
✓Network docker_default Created 0.0s
✓Container postgres Started 0.7s
PS C:\Users\Andrey\Desktop\System_booking\docker> |

PS C:\Users\Andrey\Desktop\System_booking\docker> docker-compose ps
time="2026-06-11T22:17:39+03:00" level=warning msg="C:\\Users\\Andrey\\Desktop\\System_booking\\docker\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
postgres postgis/postgis:16-3.4 "docker-entrypoint.s..." postgres About a minute ago Up About a minute 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
PS C:\Users\Andrey\Desktop\System_booking\docker>
```

Для работы с PostgreSQL будет использован DBeaver.

Создание базы данных в DBeaver:



Объявление использования расширения:

```
CREATE EXTENSION IF NOT EXISTS postgis;
SELECT PostGIS_Version();
```

Проверка, что расширение установлено (версия 3.4):

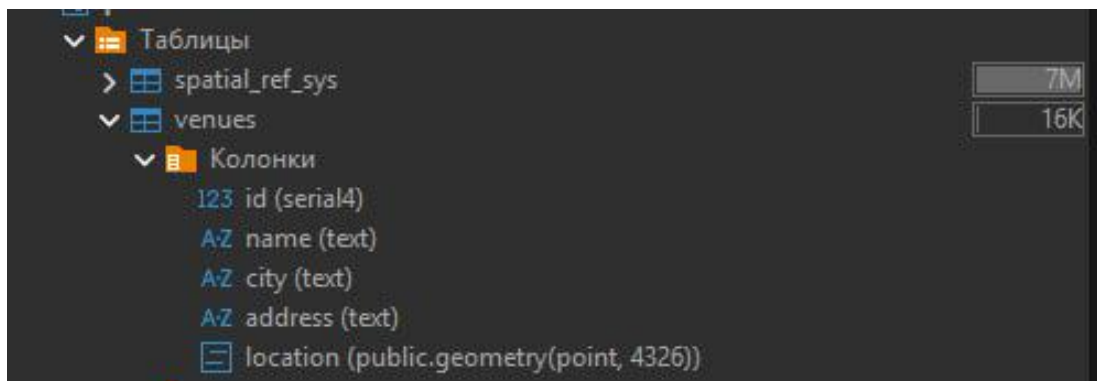
```
Text
3.4 USE_GEOS=1 USE_PROJ=1 USE_STATS=1
```

Переходим к созданию таблицы venue, её колонки: id (SERIAL PK), name, city, address, location.

Начнем с того, что location имеет тип geometry.

```
CREATE TABLE venues (
  id          SERIAL PRIMARY KEY,
  name        TEXT NOT NULL,
  city        TEXT NOT NULL,
  address     TEXT NOT NULL,
  location    GEOMETRY(Point, 4326)
);
```

Таблицы созданы:



Генерация данных в таблице venue.

Выбрана таблица из 10000 строк.

```
INSERT INTO venues (name,city,address,location)
SELECT
  'Театр №' || i,
  'Москва',
  'ул. ' ||
  (ARRAY['Пушкина', 'Ленина', 'Гагарина', 'Ломоносова', 'Кирова',
    'Пионеров'])[1 + (random()*5)::int] || ' ' || (i%100),
  ST_SetSRID(
    ST_MakePoint(
      37.6156 + random() * 0.5,
      55.7522 + random() * 0.5
    ),
    4326
```

```
)
FROM generate_series(1, 10000) AS s(i);
```

Проверяем:

```
SELECT * FROM venues LIMIT 10;
```

	123 id	AZ name	AZ city	AZ address	location
1	1	Театр №1	Москва	ул. Кирова 1	POINT (37.822549200013796 56.16318962013233)
2	2	Театр №2	Москва	ул. Кирова 2	POINT (37.797053646188346 56.07211687638208)
3	3	Театр №3	Москва	ул. Гагарина 3	POINT (37.74833631642445 56.13541015611156)
4	4	Театр №4	Москва	ул. Ленина 4	POINT (37.73451056787521 55.77171136768977)
5	5	Театр №5	Москва	ул. Ленина 5	POINT (37.755317591555624 56.134521875936244)
6	6	Театр №6	Москва	ул. Пушкина 6	POINT (38.074257871882445 56.17572930949403)
7	7	Театр №7	Москва	ул. Пионеров 7	POINT (37.728965384352584 56.072163533918946)
8	8	Театр №8	Москва	ул. Гагарина 8	POINT (37.847281040385575 56.07493609165755)
9	9	Театр №9	Москва	ул. Гагарина 9	POINT (37.967435575576886 56.14960527128809)
10	10	Театр №10	Москва	ул. Ленина 10	POINT (37.76711871693371 56.09110308022258)
11	11	Театр №11	Москва	ул. Ленина 11	POINT (37.770991793204736 55.787943411689554)
12	12	Театр №12	Москва	ул. Гагарина 12	POINT (37.77704123022129 55.87354158651438)
13	13	Театр №13	Москва	ул. Ленина 13	POINT (38.066084979682486 56.10888848530449)
14	14	Театр №14	Москва	ул. Гагарина 14	POINT (37.67694831760717 56.05347655986448)
15	15	Театр №15	Москва	ул. Гагарина 15	POINT (38.08196386081463 55.90957752680036)
16	16	Театр №16	Москва	ул. Кирова 16	POINT (37.998145972349555 55.958689865109314)
17	17	Театр №17	Москва	ул. Гагарина 17	POINT (38.01263952385589 56.04526381229933)

Посмотрим за сколько будет выполнен запрос на поиск в радиусе 10 км без использования индекса.

```
EXPLAIN ANALYZE
SELECT
  id,
  name,
  city,
  address,
  ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
WHERE ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) < 0.1
order by distance
```

Результат:

AZ QUERY PLAN
Sort (cost=169336.07..169344.51 rows=3377 width=67) (actual time=4.494..4.507 rows=307 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
Sort Method: quicksort Memory: 54kB
-> Seq Scan on venues (cost=0.00..169138.15 rows=3377 width=67) (actual time=2.512..4.432 rows=307 loops=1)
Filter: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry) < '0.1'::double precision)
Rows Removed by Filter: 9693
Planning Time: 1.561 ms
JIT:
Functions: 4
Options: Inlining false, Optimization false, Expressions true, Deforming true
Timing: Generation 0.279 ms, Inlining 0.000 ms, Optimization 0.228 ms, Emission 2.245 ms, Total 2.753 ms
Execution Time: 4.844 ms

Очевидно, использован seq scan, postgres решил еще использовать и jit, так как запрос довольно тяжелый.

Посмотрим на выполнение knn-запроса:

```
EXPLAIN ANALYZE
SELECT
  id,
  name,
  city,
  address,
  ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
ORDER BY location <-> ST_SetSRID(ST_MakePoint(37.6173,
55.7558), 4326)
LIMIT 10;
```

AZ QUERY PLAN
Limit (cost=6758.10..6889.47 rows= 10 width=75) (actual time=4.768..4.772 rows= 10 loops=1)
-> Result (cost=6758.10..138133.10 rows=10000 width=75) (actual time=4.767..4.770 rows=10 loops=1)
-> Sort (cost=6758.10..6783.10 rows=10000 width=99) (actual time=4.765..4.766 rows=10 loops=1)
Sort Key: ((location <-> '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
Sort Method: top-N heapsort Memory: 28kB
-> Seq Scan on venues (cost=0.00..6542.00 rows=10000 width=99) (actual time=0.072..0.073 rows=10 loops=1)
Planning Time: 2.714 ms
Execution Time: 4.792 ms

5 ms на вывод 10 строк – очень плохой результат. Необходим индекс.

Добавим индекс на тип GIST geometry:

```
DROP INDEX IF EXISTS idx_venues_location;
CREATE INDEX idx_venues_location ON venues USING GIST
(location);
```

Query
CREATE INDEX idx_venues_location ON venues USING GIST (location)

Теперь используем индекс GIST для поиска в радиусе.

```
EXPLAIN ANALYZE
SELECT
    id,
    name,
    city,
    address,
    ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
WHERE ST_DWithin(
    location,
    ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326),
    0.1
)
ORDER BY distance
```

Стоит отметить, что использование `ST_Distance` в `WHERE` бы не дало изменений, потому что он не работает с индексами. Поэтому заменяем на функцию `ST_DWithin`. Этот операнд будет использовать любые индексы, которые могут быть доступны.

Sort (cost=5577.72..5577.72 rows=1 width=67) (actual time=1.295..1.305 rows=307 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
Sort Method: quicksort Memory: 54kB
-> Index Scan using idx_venues_location on venues (cost=0.28..5577.70 rows=1 width=67) (actual time=0.380..1.235 rows=307 loops=1)
Index Cond: (location && st_expand('0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry, '0.1'::double precision))
Filter: st_dwithin(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry, '0.1'::double precision)
Rows Removed by Filter: 90
Planning Time: 33.832 ms
Execution Time: 1.365 ms

Индекс использовался, время выполнения уменьшилось.

Теперь сделаем запрос на поиск ближайших соседей (KNN):

```
EXPLAIN ANALYZE
SELECT
    id,
    name,
    city,
    address,
    ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
```



```

distance
FROM venues
ORDER BY location <-> ST_SetSRID(ST_MakePoint(37.6173,
55.7558), 4326)
LIMIT 10;

```

<-> операнд будет использовать n-мерные GiST-индексы, которые могут быть доступны для геометрий. Он отличается от других операторов, использующих пространственные индексы, тем, что индекс используется только тогда, когда оператор находится в блоке ORDER BY.

Limit (cost=0.15..132.38 rows=10 width=75) (actual time=0.040..0.056 rows=10 loops=1)
-> Index Scan using idx_venues_location on venues (cost=0.15..132229.15 rows=10000 width=75) (actual time=0.039..0.055 rows=10 loops=1)
Order By: (location <-> '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry)
Planning Time: 0.087 ms
Execution Time: 0.103 ms

Время выполнения очень маленькое, так как требовалось найти 10 площадок.

Так как тип geometry не является совершенно точным, когда речь идёт о реальных гео-данных, будем переходить к типу geography. Сначала через каст.

Тот же запрос, но если кастить к geography:

```

EXPLAIN ANALYZE
SELECT
    id,
    name,
    city,
    address,
    ST_Distance(
        location::geography,
        ST_MakePoint(37.6173, 55.7558)::geography
    ) AS distance_meters
FROM venues
WHERE ST_DWithin(
    location::geography,
    ST_MakePoint(37.6173, 55.7558)::geography,
    10000
)
ORDER BY distance_meters;

```

Помним, что location является типом geometry, а также что в функции параметры должны быть одного типа. В блоке WHERE можно уже указывать не градусы, а метры. Теперь запрос и фильтрация более точные.

Sort (cost=125286.51..125286.52 rows=1 width=67) (actual time= 57.442..57.470 rows=458 loops=1)
Sort Key: (st_distance((location)::geography, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, true))
Sort Method: quicksort Memory: 68kB
-> Seq Scan on venues (cost=0.00..125286.50 rows=1 width=67) (actual time=45.869..57.338 rows=458 loops=1)
Filter: st_dwithin((location)::geography, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision)
Rows Removed by Filter: 9542
Planning Time: 0.117 ms
JIT:
Functions: 4
Options: Inlining false, Optimization false, Expressions true, Deforming true
Timing: Generation 0.297 ms, Inlining 0.000 ms, Optimization 0.166 ms, Emission 2.750 ms, Total 3.213 ms
Execution Time: 57.856 ms

Почему использовался seq scan, хотя есть индекс: индекс у нас создавался под конкретный тип данных geometry, поэтому когда мы кастим значение в тип geography – в каждой строке формируется новое значение, с которым индекс не знает как работать: берется строчка – переводится в geography, проверяется – и так со всей таблицей.

Следовательно, каст – не лучший вариант. Попробуем изменить тип колонки location на geography и прогоним те же запросы.

```
ALTER TABLE venues ALTER COLUMN location TYPE
GEOGRAPHY(Point, 4326);
```

Не забываем про индекс:

```
DROP INDEX IF EXISTS idx_venues_location;
CREATE INDEX idx_venues_location ON venues USING GIST
(location);
```

ery	ALTER TABLE venues ALTER COLUMN location TYPE GEOGRAPHY(Point, 4326);
	DROP INDEX IF EXISTS idx_venues_location;
	CREATE INDEX idx_venues_location ON venues USING GIST (location)

Первый запрос:

```
EXPLAIN ANALYZE
SELECT
  id,
  name,
  city,
  address,
  ST_Distance(location, ST_MakePoint(37.6173,
```

```

55.7558)::geography) AS distance_meters
FROM venues
WHERE ST_DWithin(
    location,
    ST_MakePoint(37.6173, 55.7558)::geography,
    10000
)
ORDER BY distance_meters;

```

Первый раз:

Sort (cost=33.30..33.31 rows=1 width=67) (actual time=9.414..9.430 rows=458 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, true))
Sort Method: quicksort Memory: 68kB
-> Index Scan using idx_venues_location on venues (cost=0.28..33.29 rows=1 width=67) (actual time=8.123..9.318 rows=458 loops=1)
Index Cond: (location && _st_expand('0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision))
Filter: st_dwithin(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision, true)
Rows Removed by Filter: 273
Planning Time: 3.609 ms
Execution Time: 9.498 ms

Второй раз:

Sort (cost=33.30..33.31 rows=1 width=67) (actual time=1.592..1.615 rows=458 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, true))
Sort Method: quicksort Memory: 68kB
-> Index Scan using idx_venues_location on venues (cost=0.28..33.29 rows=1 width=67) (actual time=0.040..1.490 rows=458 loops=1)
Index Cond: (location && _st_expand('0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision))
Filter: st_dwithin(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision, true)
Rows Removed by Filter: 273
Planning Time: 0.108 ms
Execution Time: 1.644 ms

Время выполнения первого запроса во второй раз гораздо ниже – это связано с достаточно большим количеством вычислений в типе данных geography, который сложнее geometry, ведь учитывает, что Земля круглая. Когда мы выполняем запрос во второй раз – часть вычислений уже сохранена в кэше, и обработка этих значений занимает меньше времени.

Сделаем второй запрос KNN:

```

EXPLAIN ANALYZE
SELECT
    id,
    name,
    city,
    address,
    ST_Distance(location, ST_MakePoint(37.6173,
55.7558)::geography) AS distance
FROM venues
ORDER BY location <-> ST_MakePoint(37.6173,

```

```
55.7558)::geography  
LIMIT 10;
```

```
Limit (cost=0.15..128.95 rows=10 width=75) (actual time=0.074..0.107 rows=10 loops=1)  
-> Index Scan using idx_venues_location on venues (cost=0.15..128799.15 rows=10000 width=75) (actual time=0.074..0.105 rows=10 loops=1)  
    Order By: (location <-> '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography)  
Planning Time: 0.097 ms  
Execution Time: 0.166 ms
```

Время KNN почти не изменилось.

Для примера были выбраны координаты центра Москвы. В полноценном приложении должно быть реализовано взаимодействие с внешней API, которая определяла бы геоданные пользователя и направляла их в PostgreSQL. То есть примерно следующий сценарий:

- Пользователь открывает приложение, телефон/устройство даёт GPS-координаты

- Приложение делает запрос на backend (endpoint и параметры по усмотрению разработчика):

GET /venues/nearby

GET /venues/closest

- В backend, в обработчике выбранного эндпоинта, эти параметры подставляются в SQL

- Backend возвращает JSON со списком площадок и расстояниями – приложение показывает список «Ближайшие театры».

Сводная таблица по типам запросов:

Индекс	Тип столбца	Тип в запросе	Запрос «в радиусе»	Запрос KNN
-	geometry	geometry	~4.8 ms	~4.7 ms
GiST	geometry	geometry	Index Scan, ~1.3 ms	Index Scan, ~0.1 ms
GiST	geometry	geography (каст ::geography)	Seq Scan, ~57.8 ms	–
GiST	geography	geography	Index Scan, ~9.6 (~1.7) ms	Index Scan, ~0.2 ms

Выводы:

0. Поиск без индекса не оптимален.

1. GiST и geometry. Индекс используется для обоих запросов – и для поиска в радиусе, и для KNN. Это базовый рабочий вариант, но расстояния возвращаются в градусах, что неудобно для практического применения. Также имеет высокую неточность.

2. GiST и geometry с кастом к geography. Индекс не используется ни для одного запроса. PostgreSQL вынужден выполнять Seq Scan. Несмотря на то, что расстояния исчисляются в метрах, производительность падает в 45 раз.

3. GiST и geography. Индекс используется для обоих запросов, расстояния сразу в метрах. Это более медленный чем geometry вариант, но более точный и удобный для дальнейшего использования.

5. Реализация расширения системы с применением SP-GiST

Проанализируем запросы, используя индекс, реализующий quad-tree – SP-Gist.

Аналогично пройдемся по разным типам данных, начиная с геометрии.

Создание индекса:

```
DROP INDEX IF EXISTS idx_venues_location;
DROP INDEX IF EXISTS idx_venues_location_spgist;
ALTER TABLE venues ALTER COLUMN location TYPE
GEOMETRY(Point, 4326) USING
location::geometry(Point, 4326);
CREATE INDEX idx_venues_location_spgist ON venues USING
SPGIST (location);
```

Индекс на location с типом данным geometry.

Выполним 1 запрос:

```
EXPLAIN ANALYZE
SELECT id, name, city, address,
       ST_Distance(location,
                   ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
WHERE ST_DWithin(
       location,
       ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326),
       0.1
)
ORDER BY distance;
```

Sort (cost=33.30..33.31 rows=1 width=67) (actual time=0.445..0.454 rows=307 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
Sort Method: quicksort Memory: 54kB
-> Index Scan using idx_venues_location_spgist on venues (cost=0.28..33.29 rows=1 width=67) (actual time=0.058..0.380 rows=307 loops=1)
Index Cond: (location && st_expand('0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry, '0.1'::double precision))
Filter: st_dwithin(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry, '0.1'::double precision)
Rows Removed by Filter: 90
Planning Time: 3.189 ms
Execution Time: 0.511 ms

2 запрос – KNN:

```
EXPLAIN ANALYZE
SELECT
```

```

    id,
    name,
    city,
    address,
    ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
ORDER BY location <-> ST_SetSRID(ST_MakePoint(37.6173,
55.7558), 4326)
LIMIT 10;

```

```

Limit (cost=6758.10..6889.47 rows=10 width=75) (actual time=4.122..4.127 rows=10 loops=1)
-> Result (cost=6758.10..138133.10 rows=10000 width=75) (actual time=4.121..4.124 rows=10 loops=1)
-> Sort (cost=6758.10..6783.10 rows=10000 width=99) (actual time=4.119..4.120 rows=10 loops=1)
    Sort Key: ((location <-> '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
    Sort Method: top-N heapsort  Memory: 28kB
-> Seq Scan on venues (cost=0.00..6542.00 rows=10000 width=99) (actual time=0.013..2.573 rows=10000 loops=1)
Planning Time: 0.081 ms
Execution Time: 4.175 ms

```

Заметим, что использовался Seq Scan – это связано с тем, что индекс SpGist не поддерживает поиск KNN. Поэтому для KNN этот индекс не подходит.

Каст к географии:

```

EXPLAIN ANALYZE
SELECT id, name, city, address,
       ST_Distance(location::geography,
ST_MakePoint(37.6173, 55.7558)::geography) AS
distance_meters
FROM venues
WHERE ST_DWithin(
    location::geography,
    ST_MakePoint(37.6173, 55.7558)::geography,
    10000
)
ORDER BY distance_meters;

```

```

Sort (cost=125329.51..125329.52 rows=1 width=67) (actual time=15.057..15.082 rows=458 loops=1)
    Sort Key: (st_distance((location)::geography, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, true))
    Sort Method: quicksort  Memory: 68kB
-> Seq Scan on venues (cost=0.00..125329.50 rows=1 width=67) (actual time=3.591..14.930 rows=458 loops=1)
    Filter: st_dwithin((location)::geography, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision)
    Rows Removed by Filter: 9542
Planning Time: 0.087 ms
JIT:
    Functions: 4
    Options: Inlining false, Optimization false, Expressions true, Deforming true
Timing: Generation 0.278 ms, Inlining 0.000 ms, Optimization 0.193 ms, Emission 3.312 ms, Total 3.782 ms
Execution Time: 30.252 ms

```

Аналогично, как и с Gist, каст требует большего времени и не целесообразен, так как индекс создается под колонку location, имеющую тип данных geometry.

Пересоздадим индекс на тип geography:

```
ALTER TABLE venues ALTER COLUMN location TYPE
GEOGRAPHY(Point, 4326);
DROP INDEX IF EXISTS idx_venues_location_spgist;
CREATE INDEX idx_venues_location_spgist ON venues USING
SPGIST (location);
```

Выполним запрос на поиск в радиусе:

```
EXPLAIN ANALYZE
SELECT id, name, city, address,
       ST_Distance(location, ST_MakePoint(37.6173,
55.7558)::geography) AS distance_meters
FROM venues
WHERE ST_DWithin(
       location,
       ST_MakePoint(37.6173, 55.7558)::geography,
       10000
)
ORDER BY distance_meters;
```

Sort (cost=33.30..33.31 rows=1 width=67) (actual time=2.098..2.122 rows=458 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, true))
Sort Method: quicksort Memory: 68kB
-> Index Scan using idx_venues_location_spgist on venues (cost=0.28..33.29 rows=1 width=67) (actual time=0.337..2.012 rows=458 loops=1)
Index Cond: (location && _st_expand('0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision))
Filter: st_dwithin(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geography, '10000'::double precision, true)
Rows Removed by Filter: 273
Planning Time: 3.255 ms
Execution Time: 2.154 ms

Сводная таблица по типам запросов:

Индекс	Тип столбца	Тип в запросе	Запрос «в радиусе»	Запрос KNN
SP-GiST	geometry	geometry	Index Scan, ~0.5 ms	—
SP-GiST	geometry	geography (каст ::geography)	Seq Scan, ~30.2 ms	—
SP-GiST	geography	geography	Index Scan, ~2.1 ms	—

Выводы:

1. KNN запросы неприменимы.
 2. Каст к geography вновь бесполезен, так как увеличивает время выполнения и не использует индексы.
 3. Время запросов «в радиусе» значительно меньше в сравнение с Gist.
- В целом индекс немного лучше Gist, но невозможность использования в KNN может быть проблемой при реализации.

6. Использование BRIN индексов

BRIN (Block Range Index) – это тип индекса в PostgreSQL, который создаёт не запись для каждой строки таблицы, а краткую характеристику для группы физически соседних страниц данных (блоков). В контексте PostGIS он даёт возможность почти так же быстро искать пространственные данные, занимая при этом в сотни раз меньше места, чем GiST.

Для пространственных данных (геометрий) BRIN запоминает для блока страниц общий MBR, который включает в себя все геометрии из этого блока.

Рассмотрим применение этого индекса на практике:

```
DROP INDEX IF EXISTS idx_venues_location_spgist;
DROP INDEX IF EXISTS idx_venues_location_brin;
ALTER TABLE venues ALTER COLUMN location TYPE
GEOMETRY(Point, 4326) USING
location::geometry(Point, 4326);
CREATE INDEX idx_venues_location_brin ON venues USING
BRIN (location);
```

Выполним первый запрос на поиск в радиусе:

```
EXPLAIN ANALYZE
SELECT id, name, city, address,
       ST_Distance(location,
                    ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
WHERE ST_DWithin(
       location,
       ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326),
       0.1
)
ORDER BY distance;
```

Sort (cost=125304.51..125304.51 rows=1 width=67) (actual time=4.399..4.414 rows=307 loops=1)
Sort Key: (st_distance(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
Sort Method: quicksort Memory: 54kB
-> Seq Scan on venues (cost=0.00..125304.50 rows=1 width=67) (actual time=2.438..4.335 rows=307 loops=1)
Filter: st_dwithin(location, '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry, '0.1'::double precision)
Rows Removed by Filter: 9693
Planning Time: 1.667 ms
JIT:
Functions: 4
Options: Inlining false, Optimization false, Expressions true, Deforming true
Timing: Generation 0.286 ms, Inlining 0.000 ms, Optimization 0.172 ms, Emission 2.239 ms, Total 2.697 ms
Execution Time: 4.754 ms

Второй запрос на knn:

```
EXPLAIN ANALYZE
SELECT
    id,
    name,
    city,
    address,
    ST_Distance(location,
ST_SetSRID(ST_MakePoint(37.6173, 55.7558), 4326)) AS
distance
FROM venues
ORDER BY location <-> ST_SetSRID(ST_MakePoint(37.6173,
55.7558), 4326)
LIMIT 10;
```

Limit (cost=6758.10..6889.47 rows=10 width=75) (actual time=3.789..3.793 rows=10 loops=1)
-> Result (cost=6758.10..138133.10 rows=10000 width=75) (actual time=3.788..3.791 rows=10 loops=1)
-> Sort (cost=6758.10..6783.10 rows=10000 width=99) (actual time=3.787..3.787 rows=10 loops=1)
Sort Key: ((location <-> '0101000020E610000010E9B7AF03CF42408D28ED0DBEE04B40'::geometry))
Sort Method: top-N heapsort Memory: 28kB
-> Seq Scan on venues (cost=0.00..6542.00 rows=10000 width=99) (actual time=0.016..2.330 rows=10000 loops=1)
Planning Time: 0.121 ms
Execution Time: 3.808 ms

В обоих случаях индекс не был использован – использовался seq scan. Почему: BRIN хранит для каждого диапазона таблицы минимальное и максимальное значение индексируемого столбца среди строк этого диапазона. Поиск по такому индексу не эффективен так как:

1. Отсутствует связность и упорядоченность данных. Данные вставлялись случайным образом, поэтому строки имеют никак не связанные между собой координаты. В результате min/max диапазон для каждого блока страниц охватывает практически всю область, в которой расположены данные.

2. Нет одномерности представления диапазона. Для двумерных точек диапазон превращается в прямоугольник между минимальной и максимальной точкой блока – при отсутствии сортировки этот прямоугольник практически является MBR всего набора данных, что делает индекс полностью неэффективным.

ВЫВОД

В ходе работы было исследовано применение геопространственных структур в PostgreSQL для сценария поиска площадок для сервиса бронирования билетов. Расширение PostGIS было развёрнуто в Docker, создана таблица с тестовыми данными, на которой протестированы три типа индексов (GiST, SP-GiST, BRIN) для двух ключевых сценариев – поиска объектов в радиусе и поиска N ближайших объектов (KNN).

Результаты показали, что GiST, реализующий R-Tree, является единственным индексом, поддерживающим оба сценария одновременно для типов geometry и geography. SP-GiST (квадродерево) показал немного лучшую производительность для поиска в радиусе, но не поддерживает KNN – при попытке использовать оператор <-> SQL переходит на полный перебор таблицы. BRIN оказался полностью непригоден для гео-данных независимо от типа запроса, так как требует физической отсортированности данных, которая отсутствует для координат точек.

Отдельно выявлен критически важный практический фактор: соответствие типа столбца (geometry/geography) типу, используемому в условиях запроса. Несовпадение типов – например, хранение в geometry с приведением к geography прямо в запросе – приводит к тому, что построенный индекс не используется вообще, независимо от его типа, и запрос выполняется полным перебором.

На основе полученных результатов для сервиса бронирования билетов выбрана конфигурация столбец типа geography и индекс GiST – она обеспечивает корректное использование индекса для обоих сценариев геопоиска и возвращает расстояния непосредственно в метрах, притом обеспечивает точность этих результатов. Решение реализуется средствами расширения PostGIS внутри существующей OLTP-базы и может быть подключено к обработчикам без необходимости в дополнительной инфраструктуре.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Описание структуры R-Tree // Wikipedia. URL: <https://en.wikipedia.org/wiki/R-tree> (дата обращения: 11.06.2026).
2. Алгоритмы R-Tree // bartoszsypytkowski. URL: <https://www.bartoszsypytkowski.com/r-tree/> (дата обращения: 11.06.2026).
3. Документация PostgreSQL (индексы) // PostgreSQL. URL: <https://www.postgresql.org/docs/current/indexes-types.html#INDEXES-TYPE-GIST> (дата обращения: 11.06.2026).
4. Введение в R-tree // geeksforgeeks. URL: <https://www.geeksforgeeks.org/dsa/introduction-to-r-tree/> (дата обращения: 11.06.2026).
5. Структуры Gist // Wikipedia. URL: <https://en.wikipedia.org/wiki/GiST> (дата обращения: 11.06.2026).
6. Руководство Postgis // Postgis. URL: <https://postgis.net/stuff/postgis-3.4.0-en.pdf> (дата обращения: 12.06.2026).
7. Неклассические индексы // Github. URL: <https://vladtrc.github.io/content/courses/ais> (дата обращения: 12.06.2026).